

Parallel & Distributed Computing

MIDTERM EXAM COMPREHENSIVE HANDOUT

Concepts • Performance • Synchronization • Communication • Distributed Systems

BS Computer Science — April 2026

 **Midterm Exam Coverage:** This handout covers all foundational topics for the midterm examination including: Introduction to Parallelism, Flynn’s Taxonomy, Performance Metrics (Amdahl’s & Gustafson’s Laws), Parallel Algorithm Design, Process Synchronization, Inter-Process Communication, and Distributed Systems fundamentals. Study each section thoroughly, with particular attention to **formulae**, **definitions**, and **comparison tables**.

Contents

1	Introduction to Parallel and Distributed Computing	4
1.1	Motivation for Parallelism	4
1.2	Parallel vs. Distributed Computing	4
2	Flynn’s Taxonomy	5
3	Types of Parallelism	5
3.1	Data Parallelism	5
3.2	Task Parallelism	5
3.3	Pipeline Parallelism	6
4	Memory Architecture Models	6
4.1	Shared Memory Architecture	6
4.2	Distributed Memory Architecture	6
4.3	Hybrid Architecture	7
5	Performance Metrics	7
5.1	Speedup	7
5.2	Efficiency	7
5.3	Amdahl’s Law	7
5.4	Gustafson’s Law (Scaled Speedup)	8
5.5	Key Performance Metrics Summary	8

6	Parallel Algorithm Design	8
6.1	PCAM Methodology	9
6.2	Decomposition Strategies	9
6.3	Parallel Reduction	9
6.4	Load Balancing	9
7	Process Synchronization	10
7.1	Critical Section	10
7.2	Synchronization Primitives	10
7.2.1	Mutex (Mutual Exclusion Lock)	10
7.2.2	Semaphore	10
7.2.3	Monitor	11
7.2.4	Barrier	11
7.3	Deadlock	11
7.4	Livelock and Starvation	11
8	Inter-Process Communication (IPC)	11
8.1	Shared Memory IPC	11
8.2	Message Passing	12
8.2.1	Blocking vs. Non-Blocking	12
8.3	MPI (Message Passing Interface) Overview	12
8.3.1	MPI Collective Operations	12
8.4	Pipes and Sockets	13
9	OpenMP: Shared Memory Programming	13
9.0.1	Key OpenMP Directives	13
10	Distributed Systems Fundamentals	13
10.1	Goals of Distributed Systems	14
10.1.1	Types of Transparency	14
10.2	Architectural Models	14
10.2.1	Client-Server Model	14
10.2.2	Peer-to-Peer (P2P) Model	14
10.2.3	Cloud Computing Model	14
11	The CAP Theorem	15
12	Consistency Models and Replication	15
12.1	Data Replication	15
12.2	Consistency Models	15
12.3	Replication Strategies	16
13	Fault Tolerance	16
13.1	Types of Failures	16
13.2	Fault Tolerance Techniques	16
13.3	Byzantine Fault Tolerance (BFT)	17

14 Distributed Clocks and Time	17
14.1 Logical Clocks	17
15 Network Topologies for Parallel Systems	17
16 Scalability	18
17 Quick Reference: Key Formulae & Facts	19

1 Introduction to Parallel and Distributed Computing

Parallel Computing

A type of computation where many calculations or processes are performed **simultaneously**. Large problems are divided into smaller ones that are solved concurrently using multiple processors.

Distributed Computing

A model where computation is spread across multiple **networked, autonomous computers** that communicate and coordinate by passing messages to achieve a common goal.

1.1 Motivation for Parallelism

- **Performance:** Single-processor performance has plateaued due to physical limits (power wall, memory wall, ILP wall).
- **Problem Size:** Many real-world problems (weather simulation, genome analysis) are too large for a single processor.
- **Time Constraints:** Some tasks must be completed within strict deadlines (real-time systems, financial trading).
- **Cost:** Multiple commodity processors are cheaper than a single high-performance processor.
- **Reliability:** Distributed systems provide fault tolerance through redundancy.



1.2 Parallel vs. Distributed Computing

Aspect	Parallel Computing	Distributed Computing
Location	Processors at same site	Processors geographically separated
Memory	Shared or local	Each node has its own memory
Communication	Shared memory or bus	Message passing over network
Synchronization	Tight (low latency)	Loose (high latency)
Failure Model	Rare, affects whole system	Partial failures possible
Goal	Speed (performance)	Scalability & fault tolerance
Example	Multi-core CPU, GPU	Internet, Cloud, Cluster

2 Flynn's Taxonomy

💡 Flynn's Taxonomy (1966)

A classification of computer architectures based on the number of simultaneous **instruction streams** and **data streams**. Proposed by Michael J. Flynn.

	Single Data (SD)	Multiple Data (MD)
Single Instr. (SI)	SISD Single processor Sequential execution e.g., Traditional CPU	SIMD Same instruction on multiple data elements e.g., GPU, vector processing
Multiple Instr. (MI)	MISD Multiple instructions on same data e.g., Fault-tolerant pipeline	MIMD Different instructions on different data e.g., Multi-core, Clusters

- **SISD**: Classic von Neumann architecture. One processor executes one instruction at a time.
- **SIMD**: All processors execute the same instruction but on different data — ideal for data-parallel tasks (image processing, neural networks). Includes SSE/AVX extensions.
- **MISD**: Rare in practice; mainly used in safety-critical systems (multiple processors check the same data).
- **MIMD**: Most common modern parallel architecture — processors operate independently. Subdivided into:
 - *Shared Memory MIMD*: All processors access a global memory (SMP, multi-core).
 - *Distributed Memory MIMD*: Each processor has local memory; communicate via messages (cluster, MPI).

3 Types of Parallelism

3.1 Data Parallelism

📖 Data Parallelism

The same operation is applied **simultaneously to different subsets of data**. Suitable for operations that can be decomposed along data dimensions.

Example: Multiplying each element of a 1,000,000-element array by 2 — split the array across 4 cores, each handles 250,000 elements.

3.2 Task Parallelism

📖 Task Parallelism

Different tasks (operations) are executed simultaneously on the same or different data.

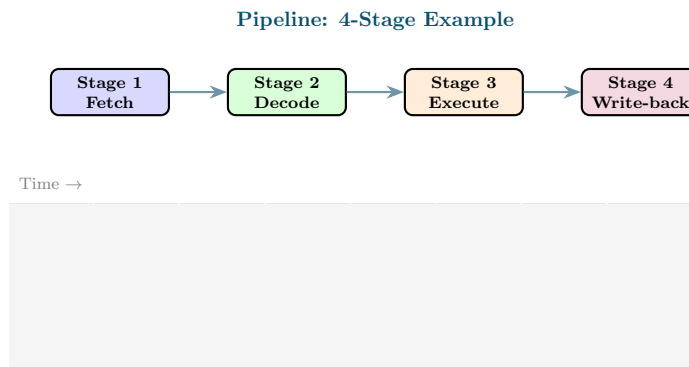
Example: While one thread downloads a file, another thread processes the already-downloaded portion.

3.3 Pipeline Parallelism

📖 Pipeline Parallelism

A computation is divided into **stages**; while stage i works on item n , stage $i-1$ already works on item $n+1$.

Example: CPU instruction pipeline (Fetch → Decode → Execute → Write-back).

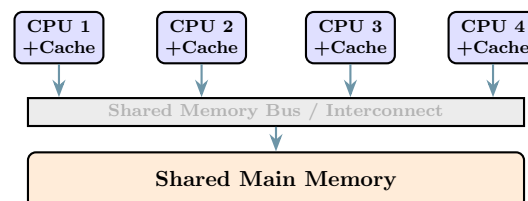


4 Memory Architecture Models

4.1 Shared Memory Architecture

💡 Shared Memory

All processors access a **single global address space**. Processors communicate by reading/writing shared variables. Fast communication but limited scalability.

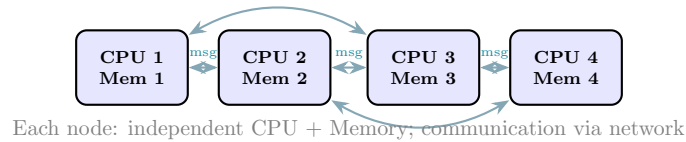


- **UMA (Uniform Memory Access):** All processors have equal memory access time. Example: SMP systems.
- **NUMA (Non-Uniform Memory Access):** Memory access time depends on the processor-to-memory physical location. Common in multi-socket servers.
- **ccNUMA:** Cache-coherent NUMA — hardware maintains cache coherency.

4.2 Distributed Memory Architecture

💡 Distributed Memory

Each processor has its **own private memory**. Processors communicate by passing messages over an interconnect network. Highly scalable, but requires explicit communication management.



4.3 Hybrid Architecture

Modern supercomputers use a **hybrid model**: multiple shared-memory nodes connected via a high-speed network (e.g., InfiniBand). Example: Each compute node has a multi-core CPU (shared memory), and nodes communicate via MPI (distributed memory).

5 Performance Metrics

5.1 Speedup

📖 Speedup Formula

$$S(p) = \frac{T_s}{T_p}$$

- T_s = Execution time on a **single processor** (serial time)
- T_p = Execution time on p processors (parallel time)
- $S(p) = p$ is **linear (ideal) speedup**
- $S(p) > p$ is called **superlinear speedup** (rare; due to cache effects)
- $S(p) < p$ is typical due to overhead

5.2 Efficiency

📖 Efficiency Formula

$$E(p) = \frac{S(p)}{p} = \frac{T_s}{p \cdot T_p}$$

- Measures how effectively processors are being utilized.
- $E = 1$ (100%): perfect efficiency (ideal)
- $E < 1$: processors sit idle or do redundant work

5.3 Amdahl's Law

⚠️ Amdahl's Law — Fundamental Limit of Parallelism

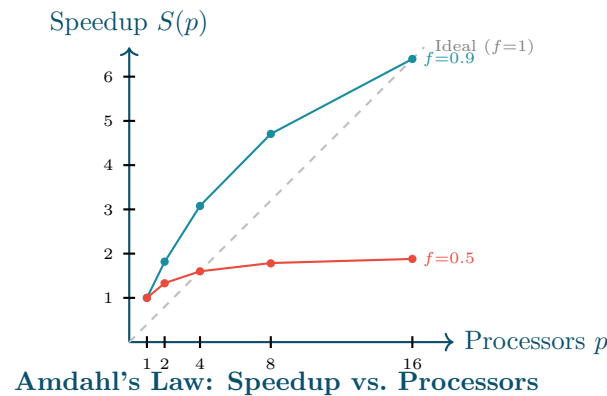
“The speedup of a program using multiple processors is limited by the sequential fraction of the program.”

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p}}$$

- f = fraction of the program that **can** be parallelized ($0 \leq f \leq 1$)
- $(1 - f)$ = fraction that must remain **serial**
- p = number of processors

- As $p \rightarrow \infty$: $S_{\max} = \frac{1}{1-f}$

Example: If 90% of a program is parallelizable ($f = 0.9$), the maximum possible speedup is $\frac{1}{1-0.9} = 10\times$, no matter how many processors are used.



5.4 Gustafson's Law (Scaled Speedup)

💡 Gustafson's Law

Unlike Amdahl's Law (which fixes problem size), Gustafson's Law assumes that as more processors are added, the **problem size scales up proportionally**.

$$S_{\text{scaled}}(p) = p - (1-f)(p-1) = f \cdot p + (1-f)$$

- f = parallel fraction of the **scaled** problem
- Implies that linear speedup *is* achievable if problem size grows with p .
- More optimistic than Amdahl's Law for real HPC workloads.

5.5 Key Performance Metrics Summary

Metric	Formula	Notes
Speedup	$S = T_s/T_p$	Ideal: $S = p$
Efficiency	$E = S/p$	Ideal: $E = 1$ (100%)
Amdahl Speedup	$S = 1/[(1-f) + f/p]$	Limit as $p \rightarrow \infty$: $1/(1-f)$
Gustafson Speedup	$S = fp + (1-f)$	Grows linearly with p
Scalability	E stays near 1 as p grows	Strong vs. Weak scaling
Communication Over-head	$T_{\text{comm}} = t_s + m/B$	t_s =latency, B =bandwidth

6 Parallel Algorithm Design

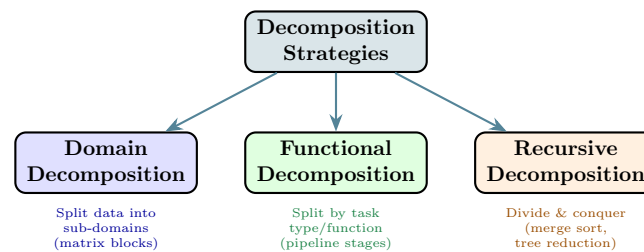
6.1 PCAM Methodology

💡 PCAM Design Steps (Ian Foster)

Partitioning → Communication → Agglomeration → Mapping

1. **Partitioning:** Break the computation into the finest-grain tasks possible (*domain decomposition* or *functional decomposition*).
2. **Communication:** Identify what data must be exchanged between tasks; determine communication patterns.
3. **Agglomeration:** Combine fine-grain tasks into larger tasks to reduce communication overhead and improve locality.
4. **Mapping:** Assign tasks to processors to maximize processor utilization and minimize communication cost.

6.2 Decomposition Strategies



6.3 Parallel Reduction

🔗 Parallel Sum (Tree Reduction)

Sum n numbers using $\log_2 n$ steps instead of $n-1$ serial additions:

Step 1: $n/2$ additions in parallel → Step 2: $n/4$ additions → ... → 1 result

Total steps: $\log_2 n$ **Work:** $O(n)$ **Span (parallel time):** $O(\log n)$

6.4 Load Balancing

- **Static Load Balancing:** Tasks are distributed before execution begins. Simple but cannot handle variable workloads.
- **Dynamic Load Balancing:** Tasks are assigned at runtime based on current processor load. More flexible; requires a scheduler (work stealing, task queues).
- **Work Stealing:** Idle processors “steal” tasks from busy processors’ queues. Used in Java ForkJoin, Intel TBB.

7 Process Synchronization

Race Condition

A situation where two or more threads/processes access **shared data concurrently**, and the final result depends on the **relative order of their execution**. This leads to **non-deterministic**, potentially incorrect behavior.

Race Condition Example

Two threads both execute `counter = counter + 1` on a shared variable:

```
// Thread 1                // Thread 2
temp1 = counter; // reads 5  temp2 = counter; // also reads 5
temp1 = temp1+1; // = 6      temp2 = temp2+1; // = 6
counter = temp1; // writes 6 counter = temp2; // writes 6 (lost
                        update!)
// Expected: counter = 7,  Actual: counter = 6
```

7.1 Critical Section

Critical Section

A segment of code that accesses **shared resources** and must **not be executed by more than one thread/process at a time**. A correct solution must satisfy:

1. **Mutual Exclusion:** Only one process in the critical section at a time.
2. **Progress:** If no process is in the critical section, one waiting process must eventually enter.
3. **Bounded Waiting:** A process must not wait forever to enter the critical section.

7.2 Synchronization Primitives

7.2.1 Mutex (Mutual Exclusion Lock)

A binary lock. A thread **acquires** (locks) the mutex before accessing shared data and **releases** (unlocks) it afterward. Only one thread can hold the mutex at a time.

7.2.2 Semaphore

Semaphore (Dijkstra)

An integer variable with two atomic operations:

- **P / wait / down:** Decrement the semaphore. If value < 0 , the process blocks.
- **V / signal / up:** Increment the semaphore. Unblocks a waiting process.

Binary semaphore (0 or 1): behaves like a mutex.

Counting semaphore: controls access to a pool of n resources.

7.2.3 Monitor

A higher-level synchronization construct that encapsulates shared data and its access procedures. Mutual exclusion is enforced automatically. Used in Java (`synchronized`), Python (`threading.Lock`).

7.2.4 Barrier

Barrier Synchronization

A synchronization point where **all processes must arrive** before any can proceed. Ensures all processes have completed a phase before the next phase begins.

Example: In parallel matrix computation, all threads finish computing row i before any starts row $i+1$.

7.3 Deadlock

Deadlock

A situation where two or more processes are **permanently blocked**, each waiting for a resource held by another.

Four Necessary Conditions (Coffman, 1971):

1. **Mutual Exclusion:** Resources cannot be shared.
2. **Hold and Wait:** A process holds at least one resource and waits for more.
3. **No Preemption:** Resources cannot be forcibly taken from a process.
4. **Circular Wait:** $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n \rightarrow P_1$ (circular dependency chain).

Prevention: Remove one of the four conditions.

Avoidance: Banker's Algorithm — grant resources only if system stays in a safe state.

Detection & Recovery: Allow deadlock, detect via resource-allocation graph, recover by termination or preemption.

7.4 Livelock and Starvation

- **Livelock:** Processes keep changing state in response to each other but make no progress (both keep “politely yielding”).
- **Starvation:** A process is perpetually denied access to a resource because others keep getting priority (e.g., low-priority thread never runs).

8 Inter-Process Communication (IPC)

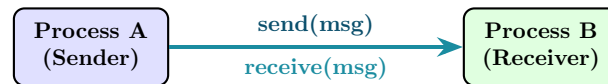
8.1 Shared Memory IPC

- Multiple processes access a **common region of memory**.
- Fastest form of IPC (no kernel involvement after setup).
- Requires synchronization (mutexes, semaphores) to prevent race conditions.
- POSIX: `shm_open()`, `mmap()`

8.2 Message Passing

💡 Message Passing

Processes communicate by sending and receiving **explicit messages**. No shared memory is needed. Used in distributed systems and between processes on separate machines.



8.2.1 Blocking vs. Non-Blocking

Type	Blocking (Synchronous)	Non-Blocking (Asynchronous)
Send	Sender waits until message is received	Sender continues immediately after sending
Receive	Receiver blocks until a message arrives	Receiver checks for message; continues if none
Advantage	Simple, no buffer needed	Higher concurrency
Risk	Can cause deadlock	Complex buffer management

8.3 MPI (Message Passing Interface) Overview

💡 MPI Key Concepts

The de-facto standard for message passing in HPC clusters. Key concepts:

- **Rank:** Unique identifier (0 to $N-1$) for each process in a communicator.
- **Communicator:** A group of processes that can communicate (MPI_COMM_WORLD = all processes).
- **Point-to-Point:** MPI_Send / MPI_Recv — one process sends to one other.
- **Collective Operations:** Involve all processes in a communicator.

8.3.1 MPI Collective Operations

Operation	MPI Call	Description
Broadcast	MPI_Bcast	Root sends data to all processes
Scatter	MPI_Scatter	Root distributes data chunks to each process
Gather	MPI_Gather	Collects data from all processes to root
Reduce	MPI_Reduce	Applies operation (sum, max) across all and sends to root
All-Reduce	MPI_Allreduce	Reduce result available at all processes
All-to-All	MPI_Alltoall	Every process sends data to every other process
Barrier	MPI_Barrier	Synchronization point for all processes

8.4 Pipes and Sockets

- **Pipe:** Unidirectional byte stream between related processes (parent-child). Named pipes (FIFOs) work between unrelated processes.
- **Socket:** Bidirectional communication endpoint. Works both locally and over a network. Supports TCP (reliable) and UDP (unreliable but fast).
- **Message Queue:** A queue of messages managed by the OS. Processes can send/receive messages asynchronously (POSIX: `mq_send`, `mq_receive`).

9 OpenMP: Shared Memory Programming

💡 OpenMP

An API for **shared-memory parallel programming** using compiler directives (`#pragma omp`), library routines, and environment variables. Supports C, C++, and Fortran.

🔗 OpenMP Parallel For Loop (C)

```
#include <omp.h>
int main() {
    int n = 1000; double sum = 0.0; double a[1000];
    // Parallel loop: iterations split across threads
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < n; i++) {
        sum += a[i]; // each thread sums its chunk
    }
    return 0;
}
```

9.0.1 Key OpenMP Directives

Directive	Effect
<code>#pragma omp parallel</code>	Creates a team of threads; each executes the block
<code>#pragma omp for</code>	Distributes loop iterations across threads
<code>#pragma omp critical</code>	Only one thread executes this section at a time
<code>#pragma omp atomic</code>	Atomic read-modify-write for a single variable
<code>#pragma omp barrier</code>	All threads wait here before proceeding
<code>reduction(op:var)</code>	Private copies reduced with <code>op</code> at end
<code>private(var)</code>	Each thread gets its own copy of <code>var</code>
<code>shared(var)</code>	All threads share the same <code>var</code>

10 Distributed Systems Fundamentals

📖 Distributed System

A collection of **independent computers** that appear to users as a single coherent system. The computers communicate only by **passing messages**.

“A distributed system is one in which the failure of a computer you didn't even know existed

can render your own computer unusable.” — Leslie Lamport

10.1 Goals of Distributed Systems

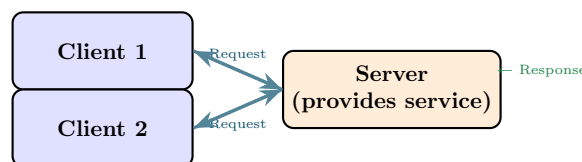
- **Transparency:** Hide the fact that resources and processes are physically distributed.
- **Openness:** Use standard protocols and interfaces; interoperability.
- **Scalability:** Handle growing number of users/resources without degradation.
- **Fault Tolerance:** Continue operating despite partial failures.
- **Performance:** Achieve better throughput/latency than a single machine.

10.1.1 Types of Transparency

Transparency Type	Description
Access	Hide differences in data representation and resource access
Location	Users don't know the physical location of resources
Migration	Resources may move without affecting users
Replication	Multiple copies exist without users knowing
Concurrency	Multiple users share resources without interfering
Failure	Hide failure and recovery from users

10.2 Architectural Models

10.2.1 Client-Server Model



- **Two-tier:** Client ↔ Server (e.g., web browser + web server).
- **Three-tier:** Client ↔ Application Server ↔ Database Server.
- **N-tier / Microservices:** Multiple layers of services.

10.2.2 Peer-to-Peer (P2P) Model

All nodes are equal (peers), acting as both **client** and **server**. No centralized server. Examples: BitTorrent, Bitcoin, Gnutella. Provides high resilience and scalability but complicates coordination.

10.2.3 Cloud Computing Model

💡 Cloud Service Models

- **IaaS (Infrastructure as a Service):** Virtual machines, storage, networking. (e.g., AWS EC2, Azure VMs)
- **PaaS (Platform as a Service):** Runtime, middleware, development tools. (e.g., Google

App Engine, Azure App Service)

- **SaaS (Software as a Service):** Ready-to-use applications. (e.g., Gmail, Microsoft 365)

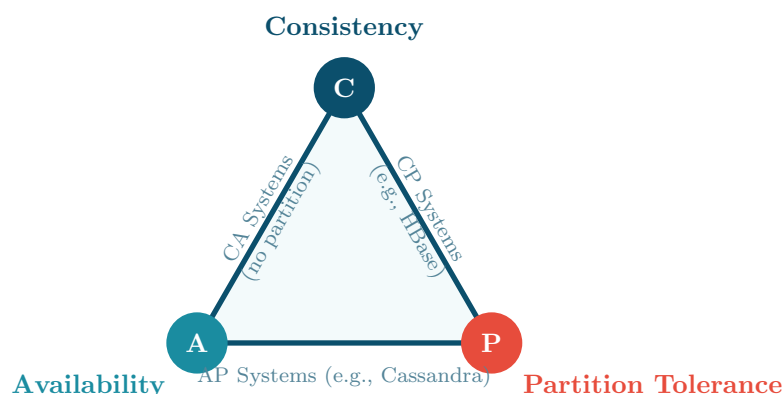
11 The CAP Theorem

⚠ CAP Theorem (Brewer's Theorem, 2000)

A distributed data store can guarantee **at most two** of the following three properties simultaneously:

1. **Consistency (C):** Every read receives the **most recent write** or an error. All nodes see the same data at the same time.
2. **Availability (A):** Every request receives a **non-error response**, even if it may not contain the most recent data.
3. **Partition Tolerance (P):** The system continues operating despite an arbitrary number of **network partition** (message loss or failure) between nodes.

Implication: In real distributed systems, network partitions *will* occur, so you must choose between C and A when partitions happen.



Trade-off	CA (no partition)	CP (consistent)	AP (available)
Guarantees	C + A	C + P	A + P
Partition	Cannot tolerate	Returns error	Returns stale data
Examples	RDBMS (single node), 2PC	Zookeeper, HBase, MongoDB (default)	Cassandra, CouchDB, DynamoDB

12 Consistency Models and Replication

12.1 Data Replication

- **Purpose:** Increase availability, fault tolerance, and read performance.
- **Challenges:** Maintaining consistency across replicas when updates occur.

12.2 Consistency Models

Model	Description
Strong Consistency	After a write completes, all subsequent reads return that value (like a single machine). Most difficult to achieve in distributed systems.
Sequential Consistency	All processes see operations in the same order, but not necessarily real-time order.
Causal Consistency	Operations that are causally related appear in the same order to all processes. Concurrent operations may differ.
Eventual Consistency	Given no new updates, replicas will eventually converge to the same value. Used by DNS, Cassandra.
Read-Your-Writes	After a process writes a value, it always reads that value or a newer one in subsequent reads.
Monotonic Read	Successive reads by the same process never return older values.

12.3 Replication Strategies

- **Primary-Backup (Master-Slave):** One primary handles all writes; replicas follow. Simple but primary is a bottleneck.
- **Multi-Primary (Multi-Master):** Multiple nodes accept writes; conflict resolution required.
- **Quorum-Based:** A write requires acknowledgment from W replicas; a read queries R replicas. If $R + W > N$ (total replicas), strong consistency is guaranteed.

13 Fault Tolerance

Fault Tolerance

The ability of a system to **continue operating correctly** in the event of one or more failures of its components.

13.1 Types of Failures

Failure Type	Description
Crash Failure	A process/node stops working and does not recover (simple to handle).
Omission Failure	A process fails to send/receive messages it should have.
Timing Failure	A process responds, but outside the required time bounds.
Response Failure	A process responds with incorrect data (wrong value or wrong action).
Byzantine Failure	A process behaves arbitrarily (may send conflicting messages). Hardest to handle.

13.2 Fault Tolerance Techniques

- **Redundancy:** Replicate components so others take over on failure (hardware, software, information redundancy).
- **Checkpointing:** Periodically save system state so it can be restored after a failure.

- **Heartbeat Monitoring:** Nodes periodically send “I’m alive” signals; absence triggers failover.
- **Majority Voting:** Use odd number of replicas; accept the value agreed upon by the majority.

13.3 Byzantine Fault Tolerance (BFT)

💡 Byzantine Generals Problem

n processes must agree on a value, but up to f of them may behave arbitrarily (send lies, different values to different nodes). Requires $n \geq 3f + 1$ processes to reach consensus despite f Byzantine failures.

14 Distributed Clocks and Time

💡 The Problem with Physical Clocks

Clocks on different machines drift and cannot be perfectly synchronized. A message sent at time t on machine A may arrive at machine B when B’s clock shows $t' < t$ — violating causality.

14.1 Logical Clocks

📖 Lamport Timestamps

Each process maintains a counter L . Rules:

1. Before a process executes an event: $L \leftarrow L + 1$.
2. When a process sends a message: include the current L in the message.
3. When a process receives a message with timestamp m : $L \leftarrow \max(L, m) + 1$.

Property: If event a *happened-before* event b (denoted $a \rightarrow b$), then $L(a) < L(b)$. But $L(a) < L(b)$ does **not** imply $a \rightarrow b$.

📖 Vector Clocks

Each process i maintains a vector $V_i[1..n]$. Rules:

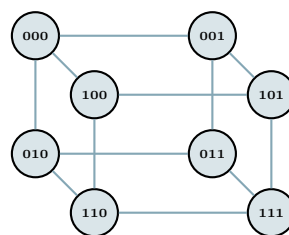
1. On local event: $V_i[i] \leftarrow V_i[i] + 1$.
2. On send: include entire vector with message.
3. On receive from j with vector V_m : $V_i[k] \leftarrow \max(V_i[k], V_m[k])$ for all k , then $V_i[i] \leftarrow V_i[i] + 1$.

Property: $V(a) < V(b)$ if and only if $a \rightarrow b$. Captures causality precisely.

15 Network Topologies for Parallel Systems

Topology	Description	Diameter	Bisection BW
Bus	All nodes on single cable	1	1 link

Topology	Description	Diameter	Bisection BW
Ring	Each node connects to 2 neighbors	$N/2$	2 links
Mesh ($\sqrt{N} \times \sqrt{N}$)	2D grid of nodes	$2(\sqrt{N} - 1)$	$\sqrt{N}$ links
Torus	Mesh with wraparound edges	$\sqrt{N}$	$2\sqrt{N}$ links
Hypercube (2^k)	k -dimensional binary cube	$k = \log_2 N$	$N/2$ links
Fat Tree	Multi-level tree; more BW at top	$O(\log N)$	$N/2$ links

Hypercube (Q_3 , 8 nodes)

16 Scalability

Scalability

The ability of a system to handle a growing amount of work or its potential to accommodate growth.

- **Strong Scaling:** The problem size is **fixed**; we add more processors to solve it faster. Limited by Amdahl's Law.
- **Weak Scaling:** The problem size **grows proportionally** with the number of processors; goal is to maintain constant execution time. Related to Gustafson's Law.
- **Horizontal Scaling (Scale Out):** Add more machines to a distributed system.
- **Vertical Scaling (Scale Up):** Add more resources (CPU, RAM) to existing machines.

Weak Scaling Efficiency

$$E_{\text{weak}} = \frac{T_1}{T_p}$$

where T_1 = time for problem of size n on 1 processor and T_p = time for problem of size $n \cdot p$ on p processors. Ideal: $E_{\text{weak}} = 1$.

17 Quick Reference: Key Formulae & Facts

★ Exam Formula & Fact Sheet

Performance Metrics:

- Speedup: $S = T_s/T_p$
- Efficiency: $E = S/p$
- Amdahl's: $S = \frac{1}{(1-f) + f/p}$
- Max Speedup (Amdahl): $S_{\max} = \frac{1}{1-f}$
- Gustafson's: $S = fp + (1-f)$

Key Numbers to Remember:

- Flynn's: SISD, SIMD, MISD, MIMD
- Deadlock: 4 Coffman conditions
- CAP: Choose 2 of 3 (C, A, P)
- Byzantine: need $n \geq 3f + 1$ nodes
- Hypercube 2^k : diameter = k
- Semaphore ops: P (wait/down), V (signal/up)
- Lamport: $a \rightarrow b \Rightarrow L(a) < L(b)$

☰ Comparison Summary

Topic	Parallel	Distributed
Memory	Shared	Private (each node)
Communication	Shared variables	Message passing
Failure	Rare, total	Common, partial
Synchronization	Mutex, semaphore	Distributed protocols
Scalability	Limited (100s CPUs)	High (1000s nodes)
Example	OpenMP, CUDA	MPI, Hadoop, Cloud

— End of Handout — BS CS | Parallel & Distributed Computing | Midterm Exam | April 2026 —